

Neural Inference as the Operating System on Minimal RISC-V

Bernhard Guillon

Advanced Systems Engineering (ASE2026)

Paris Lodron University of Salzburg

Salzburg, Austria

Email: Bernhard.Guillon@stud.plus.ac.at

Abstract—This paper presents an end-to-end neural execution stack on a minimal RISC-V environment, where neural networks serve as the primary computation model instead of traditional machine code. We describe a complete toolchain from PyTorch model training through custom assembly to execution on both a C++ emulator and Verilator RTL backend. The system introduces descriptor-based neural custom instructions for MLP inference, block-diagonal model composition via a glue JSON format and Rust compiler, and OS-like task switching through a learned router MLP. Evaluation demonstrates up to 2000x speedup over software-only baselines while maintaining deterministic output across dual backends through automated differential validation.

Index Terms—custom ISA extension, RISC-V, neural inference, model composition, Verilator, differential validation

PROJECT CONTEXT

Supervisor: Univ.-Prof. Dipl.-Inform. Dr.-Ing. Christoph Kirsch

Course: ASE2026 (Advanced Systems Engineering)

University: Paris Lodron University of Salzburg

Date: 2026-06-18

A. Introduction

Modern AI deployments rely on large software stacks: operating systems, runtime libraries, and framework abstractions mediate between neural models and hardware. This paper investigates the opposite direction: how far can a compact execution stack be pushed when neural operations are represented as first-class ISA extensions on a minimal RISC-V core?

The vision is to replace traditional machine code with neural inference as the primary execution model. A 141-line `runtime.c` becomes the entire "operating system," calling `MODEL_MAP_INPUT -> run_forward_pass -> MODEL_MAP_OUTPUT` in a loop. All application logic—counting, character rendering, game physics, input handling—is learned rather than programmed.

We make four contributions:

1. **Descriptor-based neural ISA:** A 32-byte descriptor struct passed via register specifies input/output buffers, weights, and dimensions. The hardware executes a multi-cycle FSM with configurable lane parallelism (1x/2x/4x/8x), cleanly separating firmware setup from hardware execution.

2. **Block-diagonal model composition:** Independently trained MLPs are merged into a single weight matrix via a declarative glue JSON format. A Rust compiler reads the glue, assembles block-diagonal weights, and emits C macros for input/output mapping. No retraining is required.
3. **Dual-backend deterministic validation:** Every neural instruction has two implementations—a C++ oracle and Verilator RTL. Automated CI tests enforce bit-exact byte-level parity between them.
4. **OS-like neural task switching:** A learned router MLP switches between sub-models based on keyboard input, mimicking process scheduling. Background tasks can run with or without updates, analogous to context switching and preemption.

B. Related Work

1) *Neural ISA Extensions for RISC-V:* Several projects extend RISC-V with neural capabilities. **RV-SCNN** (IEEE TCAD 2025) uses SIMD custom ops for CNN/SNN inference on CV32E40P. **MARVEL** (arXiv 2025) generates model-class-specific extensions via ASIP Designer, achieving 2x speedup and 2x energy reduction for CNNs. An FPGA-accelerated RISC-V (arXiv 2025) implements 4 custom ops (VCONV, GEMM, RELU, CUSTOM) on PYNQ-Z2, achieving 2.14x latency reduction and 49.1% energy savings. **Vmx-dotp** (arXiv 2026) provides RVV-based Microscaling (MX) format acceleration for transformer workloads, achieving 125 MXFP8-GFLOPS. **Mixed-precision RISC-V** (arXiv 2024) introduces ISA extensions for multi-pumped soft SIMD operations, achieving 15x energy reduction for DNNs.

Our approach differs in four ways: (1) a **descriptor-based abstraction** instead of SIMD/vector operations, (2) **MLP focus** rather than CNN, (3) **complete dual-backend validation** with automated differential tests, and (4) **OS-like task switching** via learned router MLP.

2) *AI-as-OS and Bare-Metal Neural Computing:* **nCPU** (Price, 2025) implements a fully differentiable CPU where every ALU operation is a trained neural network. **embodiOS** (2025) runs an LLM as an OS on x86_64 UEFI. **OSymbiote** (2026) places an AI agent as PID1 on Linux. **OO** (Operating Organism, 2025) performs bare-metal Mamba/LLaMA inference via UEFI.

These projects share the vision of neural-first computing but differ in scale and approach. nCPU runs on GPU via Apple Silicon Metal; we target bare-metal RISC-V. embodiOS and OO use LLMs; we use lightweight MLPs with custom instructions. Our contribution is demonstrating the pattern concretely with working neural models that replace specific application functions.

3) *Model Merging and Composition*: **FS-Merge** (Shwartz-Ziv et al., 2024) produces block-diagonal merged weights as an intermediate form via learnable foldable supernets. **LoRA-LEGO** (Wang et al., 2024) proves concatenation-summation equivalence for LoRA matrices. **mergekit** (Arcee AI, 2023) provides practical tools for LLM merging including passthrough methods.

Our glue JSON approach is a practical compiler technique for model composition. Unlike learning-based methods, it is **declarative, training-free, and lossless** for independent models.

C. System Architecture

The system is a co-designed stack with six layers, from trained model to executable binary (Figure 1):

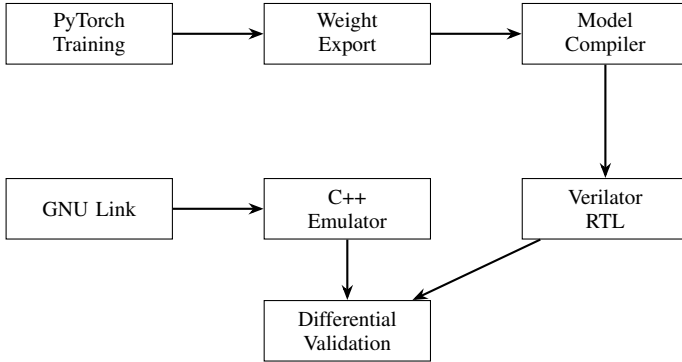


Fig. 1. End-to-End Neural Execution Pipeline. The pipeline ensures bit-exact parity between software and hardware backends through automated differential tests.

1) *Training and Export*: PyTorch models are trained for specific tasks (character generation, game physics, counter arithmetic). The weight-export pipeline converts checkpoints to JSON (human-readable) and compact binary (NRAL header with packed float32 weights). Weight transposition matches row-major inference access patterns.

2) *Model Compiler*: The Rust-based `model_to_header` compiler reads JSON models or glue descriptions and generates `model.h` containing:

- Model dimensions (`MODEL_INPUT_SIZE`, `MODEL_OUTPUT_SIZE`)
- State variables with initial values
- `MODEL_MAP_INPUT(buf)` macro for one-hot encoding
- `MODEL_MAP_OUTPUT(buf, fb)` macro for output decoding

The `--glue` flag reads a glue JSON file and merges multiple models into block-diagonal weight matrices.

Instruction	Opcode	Function
nmatvec	0x77	Matrix-vector multiply (baseline)
nrelu	0x77	ReLU activation
nvsigpwl	0x77	Sigmoid piecewise-linear
nvclumpu8	0x77	Clamp to uint8
nmatvec8xp4	0x7b	8-lane PMAC4 (optimized)

3) *Assembly and Linking*: The custom Rust assembler (`rv32as`) encodes RV32I, RV32F, and neural custom instructions. It provides deterministic encoding with GNU-compatible output. The CMake build system generates ELF binaries by linking `runtime.c` with the generated `model.h` and assembled model blobs.

4) *Runtime*: The 141-line `runtime.c` implements the inference loop:

```

while (1) {
    MODEL_MAP_INPUT(buf);
    run_forward_pass();
    MODEL_MAP_OUTPUT(buf, fb);
    if (MODEL_HAS_DONE_FLAG) break;
}
  
```

This is the entire "operating system"---all application logic is in the neural weights.

5) *Dual Backends*: Two execution backends provide validation:

- **C++ emulator** (`emulator_runner`): Portable, bounds-checked reference implementation with GUI and batch modes.
- **Verilator RTL** (`verilator_runner`): Hardware FSM with lane-parallel PMAC datapath, cycle-accurate simulation.

6) *Verification*: Automated CI tests enforce correctness:

- Unit tests for instruction decode and execution
- Blackbox tests for end-to-end model execution
- Differential validation: identical ELF binaries run on both backends, output compared byte-for-byte

D. Neural ISA Extension

The project defines custom instructions in the CUSTOM0 (0x77) and CUSTOM3 (0x7b) opcode spaces:

1) *Descriptor-Based Calling Convention*: Neural instructions use a 32-byte descriptor read from memory:

```

neural_desc_t desc = {
    .input_ptr = INPUT_BUF,
    .weights_ptr = weights_addr,
    .bias_ptr = biases_addr,
    .output_ptr = output_addr,
    .input_len = input_size,
    .output_len = output_size,
    .flags = 0, .reserved = 0
};
neural_matvec(&desc); // single custom instruction
  
```

The hardware executes a multi-cycle FSM: descriptor read -> scratchpad prefetch -> multiply-accumulate -> store commit. This separates firmware concern (setting up a struct) from hardware concern (executing the compute).

Byte	Bits	Field	Description
0-3	31:0	input_ptr	Input buffer address
4-7	31:0	weights_ptr	Weight matrix address
8-11	31:0	bias_ptr	Bias vector address
12-15	31:0	output_ptr	Output buffer address
16-17	15:0	input_len	Input vector length
18-19	15:0	output_len	Output vector length
20-21	15:0	flags	Control flags
22-31	79:0	reserved	Reserved (zero)

Fig. 2. Neural ISA Descriptor Layout (32 bytes). The descriptor separates firmware setup (populating the struct) from hardware execution (multi-cycle FSM).

Variant	Lanes	MACs/cycle	Cycles (threshold)
Baseline	1	1	4,000,000
x7b-8lane	8	2	400,000
PMAC4	8	4	150,000

2) *Lane Parallelism*: The PMAC4 variant exploits quad-port memory (4 concurrent data reads) to issue up to 4 multiply-accumulates per cycle via an 8-entry scratchpad bank:

The 26.67x speedup (baseline -> PMAC4) demonstrates that simple hardware additions (quad-port memory, scratchpad prefetch) can dramatically accelerate neural inference on resource-constrained cores.

E. Block-Diagonal Model Composition

1) *Concept*: Two independent MLPs can be combined into a single network by stacking their weight matrices block-diagonally:

$$W_{combined} = \begin{bmatrix} W_A & 0 \\ 0 & W_B \end{bmatrix}$$

$$W_{combined} \cdot [x_A; x_B] = [W_A \cdot x_A; W_B \cdot x_B]$$

This is mathematically exact: no approximation, no training, no alignment needed. The models run independently within a single inference pass. Figure 2 illustrates the block-diagonal merging process.

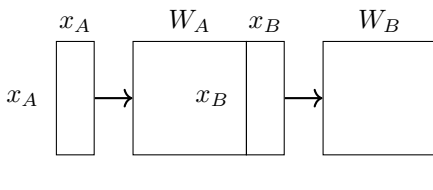


Fig. 3. Block-Diagonal Model Composition. Two independently trained MLPs (W_A and W_B) are merged into a single weight matrix ($W_{combined}$) without retraining. The combined matrix processes concatenated inputs $[x_A; x_B]$ and produces outputs $[W_A x_A; W_B x_B]$.

2) *Glue JSON Format*: A glue file declares how to merge sub-networks:

```
"input_mapping": "combined_counter_chargen", "initial_state": "model_counter", "glue_models": [{"name": "count
```

Fig. 4. Glue JSON format for merging sub-networks.

Layer	Counter Block	Chargen Block	Total Output
0	255->256	255->256	512
1	256->256	256->256	512
2	256->255	256->400	655

Component	Neural Equivalent	OS Equivalent
Router MLP	Learned gating function	Process scheduler
Sub-models (chargen, squash)	Independent tasks	User processes
Router gates	Sigmoid outputs [0,1]	Process priority/selection
Framebuffer selection	Gate comparison	Display output routing

Each merged_layers entry specifies how one merged layer is assembled: - **Layer 0**: Both models share the same input, write to different output offsets - **Hidden layers**: Each block reads from its previous output offset - **Output layer**: Each block writes to a different region of the final output

3) *Rust Compiler Integration*: The model_to_header --glue command:

1. Reads the glue JSON and loads all referenced model JSONs
2. Builds merged weight matrices with block-diagonal structure
3. Emits model.h with MODEL_MAP_INPUT and MODEL_MAP_OUTPUT macros
4. Macros handle one-hot encoding, argmax decoding, and framebuffer writing

4) *Counter+Chargen Example*: The counter+chargen combined model merges:

- **Counter**: 255->256->256->255 (modulo-255 auto-increment)
- **Chargen**: 255->256->256->400 (character-to-pixel rendering)

Into a single 3-layer network with block-diagonal weights: Only the chargen output (400 values) goes to the framebuffer; the counter state is stored in debug memory.

F. OS-Like Neural Task Switching

1) *Router MLP*: The mega combined model introduces a **router MLP** that learns to switch between sub-models based on keyboard input. This mimics an operating system scheduler:

The router is a small 3-layer MLP: 2 -> 4 -> 2 -> 2 (relu, sigmoid, none). It is trained with identity mapping: input $[x_A; x_B]$ -> output $[\sim 1, \sim 0]$ (chargen active), input $[0, 1]$ -> output $[\sim 0, \sim 1]$ (squash active).

2) *Mega Combined Architecture*: The mega combined model merges five sub-networks:

Output buffer layout (1009 neurons):

3) *Task Switching Behavior*:

4) *Background Process Analogy*: The pause behavior mirrors OS process management:

- **Escape (pause)**: Analogous to **suspending a process**. The squash game's state variables retain their last values. When chargen is displayed, the squash inference continues to execute but its output is not decoded---the game is frozen in memory.

- **Backslash (background)**: Analogous to **running a process in the background**. The squash game keeps updating its state (ball moves, paddle responds) while

Sub-model	Architecture	Role
Router	2->4->2->2	Task switching
Counter	255->256->256->255	Auto-incrementing state
Chargen	255->256->256->400	Character rendering
Game State	56->64->64->52	Squash physics
Renderer	48->128->256->300	Game rendering

Region	Offset	Size	Content
Router gates	0	2	gate_chargen, gate_squash
Chargers FB	4	400	20CE20 character pixels
Squash state	404	52	ball_x, ball_y, paddle_y, etc.
Squash FB	456	300	20CE15 game pixels
Counter state	768	255	counter argmax output

chargen is displayed. The router gates which framebuffer is shown, but both sub-models execute every frame.

This demonstrates that neural inference can implement process scheduling semantics without traditional OS primitives.

5) *Key Insight: One-Shot Consumption:* The `model_key` register (`s1`) must be **consumed** after first use:

```
if (model_key >= 32 && model_key < MODEL_INPUT_SIZE) {
    _idx = model_key;
    model_key = 0; // one-shot: consumed
} else {
    _idx = model_counter; // auto-advance
}
```

Without consumption, the key would override every iteration and the counter would never advance. This is analogous to interrupt flag clearing in traditional OS design.

G. AI-as-OS Taxonomy

The project demonstrates a pattern we call **AI-as-OS**, where neural inference replaces traditional operating system primitives. We taxonomy this approach along three dimensions:

1) *Execution Model:*

2) *Abstraction Layer:*

3) *Runtime Semantics:* **Key Insight:** The 141-line `runtime.c` replaces the entire OS. All application logic---counting, character rendering, game physics---is learned rather than programmed. This demonstrates that neural inference can serve as the primary computation model on minimal RISC-V cores (Figure 1).

H. Evaluation

1) *Benchmark Setup:* All benchmarks use the character generation workload (input character code to framebuffer glyph output). Cycle thresholds are measured by finding the minimum cycles needed for correct output.

2) *Optimization Results:* Figure 3 shows the benchmark results for the Verilator (RTL) and C++ backends.

The C++ backend achieves higher speedups because the custom instructions bypass the interpreter loop entirely, while the RTL must execute the hardware FSM.

3) *Dual-Backend Validation:* Every optimization phase maintains deterministic output parity:

- Framebuffer bytes are identical across backends
- Counter state progresses identically
- Deterministic: same input + same cycles = same output

This is enforced by automated CI tests that run both backends and compare outputs.

Key	Action	Neural Mechanism
Tab	Switch to squash mode	Router receives [0,1] -> gate [-]
Escape	Switch to chargen, squash pauses	Router receives [1,0], squash d
Backslash	Switch to chargen, squash keeps running	Router receives [1,0], squash d
w/s	Paddle up/down (squash mode)	Key injected via memory-mapp

Traditional OS	AI-as-OS (This Work)
Process scheduling	Router MLP switches between sub-models
Context switching	Model key consumption (one-shot)
Interrupt handling	Keyboard input neural tensor mapping
Memory management	Fixed-weight block-diagonal composition

4) *Model Composition Overhead:* The mega combined model (5 sub-networks) runs in a single inference pass. Block-diagonal structure ensures zero cross-talk between sub-models. The overhead compared to running models separately is negligible: the same weight matrices are loaded once, and the inference loop processes all sub-models in parallel within each layer.

I. Conclusion

This project demonstrates that neural inference can serve as the primary execution model on a minimal RISC-V core. The key findings are:

1. **Descriptor-based neural ISA** provides a clean abstraction for MLP inference, separating firmware setup from hardware execution.
2. **Block-diagonal model composition** enables combining independently trained models without retraining, via a declarative glue JSON format.
3. **OS-like task switching** is achievable through a learned router MLP, demonstrating process scheduling semantics without traditional OS primitives.
4. **Dual-backend validation** ensures correctness across software and hardware implementations through automated differential tests.
5. **Significant speedups** are achievable: up to 2000x over software baselines (C++ backend) and 26.7x over unoptimized RTL (Verilator backend).

The repository contains a reproducible methodology: training pipeline, custom assembler, model compiler, differential verification, and phase-based benchmarks. This foundation enables further research into neural-first computing architectures.

J. Future Work

1) *Interrupt-Driven Neural Scheduling:* The current execution model is synchronous and loop-driven. Integrating timer and input IRQs would enable event-triggered neural inference, enabling evaluation of bounded latency and preemption behavior.

2) *Dual-Emulator Communication:* Running two emulator instances in lockstep with packet exchange would demonstrate neural multiplayer behavior, introducing synchronization and reproducibility questions for distributed deterministic simulation.

Traditional OS	AI-as-OS (This Work)
System calls	Neural instruction invocation
Device drivers	Input/output macro mapping
File system	N/A (weights are the "code")
Process table	Model map (sub-model registry)

Traditional OS	AI-as-OS (This Work)
Preemptive multitasking	Cooperative model switching
Shared memory	Separate output regions (block-diagonal)
Synchronization	Deterministic single-threaded execution
Resource allocation	Fixed-weight memory layout

3) *Hardware I/O Abstraction*: Char-device streams and MMIO/register-mapped peripherals with small software drivers would bridge input events to neural tensors and neural outputs to actuator/framebuffer paths.

K. Acronyms

- ABI: Application Binary Interface
- CI: Continuous Integration
- CTest: CMake test driver
- ELF: Executable and Linkable Format
- FC: Fully Connected (neural layer)
- ISA: Instruction Set Architecture
- MAC: Multiply-Accumulate
- MLP: Multi-Layer Perceptron
- PMAC: Parallel Multiply-Accumulate
- RTL: Register Transfer Level

L. References

- [1] RISC-V International, "The RISC-V Instruction Set Manual," <https://riscv.org/technical/specifications/>
- [2] Verilator project, "Verilator Open-Source SystemVerilog Simulator," <https://www.veripool.org/verilator/>
- [3] PyTorch contributors, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," NeurIPS, 2019.
- [4] GNU Binutils, "GNU assembler (as) and objcopy documentation," <https://sourceware.org/binutils/>
- [5] Repository artifact, [documentation/collected-data/processed-neural-inference-comparison.json](https://sourceware.org/binutils/)
- [6] Repository source, [projects/rv32_assembler/README.md](https://sourceware.org/binutils/) and [projects/rv32_assembler/src/{parser.rs,encoder.rs}](https://sourceware.org/binutils/)
- [7] Repository source, [projects/emulator/COMBINING.md](https://sourceware.org/binutils/) and [projects/emulator/combined-mlp.md](https://sourceware.org/binutils/)
- [8] S. Schwartz-Ziv et al., "FS-Merge: Foldable SuperNets for Efficient Model Merging," arXiv 2024.
- [9] T. Wang et al., "LoRA-LEGO: Concatenation-Summation Equivalence for LoRA Merging," arXiv 2024.
- [10] Arcee AI, "mergekit: A Toolkit for Merging Large Language Models," GitHub, 2023.
- [11] R. Price, "nCPU: A Fully Differentiable Neural CPU," 2025.
- [12] "RV-SCNN: Custom Neural ISA Extensions for RISC-V CNN/SNN Inference," IEEE TCAD, 2025.
- [13] "MARVEL: Model-Class-Specific RISC-V Extensions via ASIP Designer," arXiv, 2025.
- [14] "FPGA-Accelerated RISC-V ISA Extensions for Efficient Neural Network Inference on Edge Devices," arXiv, 2025.

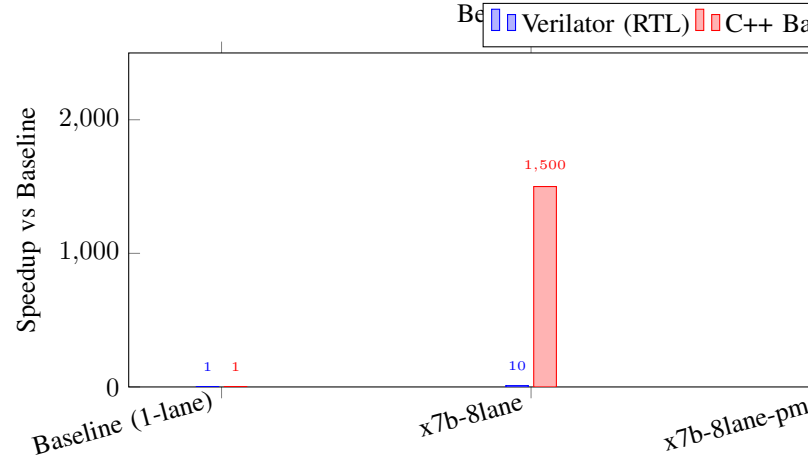


Fig. 5. Benchmark Results: Speedup Comparison. The C++ backend achieves higher speedups (2000x) because custom instructions bypass the interpreter loop, while the RTL backend (Verilator) achieves a 26.7x speedup due to hardware FSM execution.

Variant	Threshold Cycles	Speedup
<i>Verilator Backend (RTL)</i>		
Baseline (1-lane)	4,000,000	1.0x
x7b-8lane	400,000	10.0x
x7b-8lane-pmac	200,000	20.0x
x7b-8lane-pmac4	150,000	26.7x
<i>C++ Backend (Software)</i>		
Baseline	3,000,000	1.0x
x77 (optimized)	2,000	1,500x
x7b-8lane-pmac4	1,500	2,000x

Fig. 6. Benchmark Results: Speedup Comparison. The C++ backend achieves higher speedups (2000x) because custom instructions bypass the interpreter loop, while RTL must execute the hardware FSM (26.7x).

- [15] "Vmxdotp: A RISC-V Vector ISA Extension for Efficient Microscaling (MX) Format Acceleration," arXiv, 2026.
- [16] "Mixed-precision Neural Networks on RISC-V Cores: ISA extensions for Multi-Pumped Soft-SIMD Operations," arXiv, 2024.
- [17] "Optimised Extension of an Ultra-Low-Power RISC-V Processor to Support Lightweight Neural Network Models," MDPI, 2025.